شرح سطر بسطر ١٤ — حسابي (19) ورمز الطالب (29)

مشروع رِزمة · Flutter · المرحلة 6 · كل سطر مع سبب كتابته بهذه الطريقة

شاشتان: حسابي (أربع مجموعات: الملف، الدراسة، التنزيل والتخزين، عام) ورمز الطالب (الرمز مع نسخ ومشاركة، وسبب وجوده، وحالة الحفظ على الإنترنت). الودجتات المشتركة والأوراق والتفضيلات في الملف التالي (١٥).

كل رقم ظاهر حقيقي: النجوم من مجموع `starsEarned` في المحاولات، والرمز السري «مفعّل» إن وُجدت تجزئته في الملف، وعدد النتائج المنتظرة من عدد صفوف طابور المزامنة. لا رقم منسوخ من Figma.

قرار مقصود: «الوضع الليلي» معطّل مع كلمة «قريبًا» لأن الثيم الداكن غير مبني ومفتاح لا يفعل شيئًا يخدع الطالب. ومفاتيح «أصوات الألعاب» و«Wi-Fi فقط» و«تنزيل الفيديو» تُحفظ الآن وتُقرأ حين تُبنى ميزاتها؛ وتقول الواجهة ذلك في ملاحظة «يعمل مع التنزيلات عند توفرها».

لا نتحقق من الرمز الحالي قبل تغييره أو إلغائه من حسابي، لأن الطالب أصلًا دخل ملفه بعد إدخاله (أو لأنه بلا رمز). هذا قرار مقصود ومذكور في ملف الشرح الموضوعي.

lib/app/tab_navigation.dart — ماذا يحدث عند ضغط تبويب

lib/app/tab_navigation.dart

الأسطر 1–2

```
1  import 'package:flutter/material.dart';
2  import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material (لـ `BuildContext` و`ScaffoldMessenger`) وgo_router.

لماذا بهذه الطريقة؟ نحتاج `context.go` للانتقال.

الأسطر 4–6

```
4  import '../core/strings/ar.dart';
5  import '../core/widgets/bottom_nav.dart';
6  import 'routes.dart';
```

ماذا يفعل؟ استيراد النصوص والشريط السفلي (لتعداد `RizmaTab`) والمسارات.

لماذا بهذه الطريقة؟ الدالة تعرف المسارات فلا تعرفها الشاشات؛ الشاشة تقول فقط أي تبويب اختير.

الأسطر 8–9

```
8  /// ما يحدث عند ضغط تبويب في الشريط السفلي. شاشات «المواد» و«تقدّمي» لم تُبنَ
9  /// بعد، فنخبر الطالب بدل أن نتجاهل الضغطة بصمت.
```

ماذا يفعل؟ تعليق: شاشتا «المواد» و«تقدّمي» لم تُبنيا بعد.

لماذا بهذه الطريقة؟ نشرح سبب الفرع الأخير: نخبر الطالب بدل تجاهل ضغطته بصمت.

السطر 10

```
10  void onTabSelected(BuildContext context, RizmaTab current, RizmaTab target) {
```

ماذا يفعل؟ دالة `onTabSelected(context, current, target)` دالة حرة (ليست صنفًا).

لماذا بهذه الطريقة؟ منطق بلا حالة؛ تُستدعى من الرئيسية وحسابي بنفس الطريقة فلا نكرره.

السطر 11

```
11    if (target == current) return;
```

ماذا يفعل؟ إن كان التبويب المختار هو الحالي لا نفعل شيئًا.

لماذا بهذه الطريقة؟ إعادة فتح الشاشة نفسها تعيد بناءها بلا فائدة وتُفقد موضع التمرير.

الأسطر 12–14

```
12    switch (target) {
13      case RizmaTab.home:
14        context.go(AppRoutes.home);
```

ماذا يفعل؟ `switch` على الهدف: «الرئيسية» ← `context.go(home)` .

لماذا بهذه الطريقة؟ `go` يستبدل مكدس الصفحات؛ التبويبات مستوى واحد لا تتراكم فوق بعضها.

الأسطر 15–16

```
15      case RizmaTab.profile:
16        context.go(AppRoutes.account);
```

ماذا يفعل؟ «حسابي» ← `context.go(account)` .

لماذا بهذه الطريقة؟ الشاشة الفرعية (رمز الطالب) هي التي تُفتح بـ `push` .

الأسطر 17–21

```
17      case RizmaTab.subjects:
18      case RizmaTab.progress:
19        ScaffoldMessenger.of(
20          context,
21        ).showSnackBar(const SnackBar(content: Text(ArStrings.navSectionSoon)));
```

ماذا يفعل؟ «المواد» و«تقدّمي»: نعرض `SnackBar` برسالة «قيد الإعداد».

لماذا بهذه الطريقة؟ حالتان تشتركان في فرع واحد (سقوط case في Dart 3 مسموح لحالة فارغة). رسالة صادقة بدل زر ميّت.

الأسطر 22–23

```
22    }
23  }
```

ماذا يفعل؟ إغلاق `switch` والدالة.

لماذا بهذه الطريقة؟ `switch` شاملة على التعداد فيحذّرنا المحلّل لو أضفنا تبويبًا جديدًا ونسينا معالجته.

lib/features/account/account_providers.dart — مزوّدات حسابي ورمز الطالب

lib/features/account/account_providers.dart

الأسطر 1–2

```
1  import 'package:flutter_riverpod/flutter_riverpod.dart';
2  import 'package:share_plus/share_plus.dart';
```

ماذا يفعل؟ استيراد Riverpod وحزمة `share_plus`.

لماذا بهذه الطريقة؟ `share_plus` تفتح نافذة المشاركة الأصلية في النظام.

الأسطر 4–6

```
4  import '../../core/db/database_provider.dart';
5  import '../profiles/active_profile_provider.dart';
6  import 'preferences_repository.dart';
```

ماذا يفعل؟ استيراد مزوّد القاعدة ومزوّد الملف النشط ومستودع التفضيلات.

لماذا بهذه الطريقة؟ ما نحتاجه لبناء المزوّدات الجديدة.

الأسطر 8–10

```
8  final preferencesRepositoryProvider = Provider<PreferencesRepository>((ref) {
9    return PreferencesRepository(ref.watch(appDatabaseProvider));
10 });
```

ماذا يفعل؟ `preferencesRepositoryProvider`: ينشئ `PreferencesRepository` على القاعدة.

لماذا بهذه الطريقة؟ مزوّد لا متغيّر عام فيُستبدل في الاختبارات ويتشارك القاعدة نفسها مع بقية المستودعات.

الأسطر 12–20

```
12 /// تفضيلات الملف النشط (تتحدّث فور تغييرها). بلا ملف نشط: الافتراضيات.
13 final activePrefsProvider = StreamProvider<ProfilePrefs>((ref) async* {
14   final id = await ref.watch(activeProfileIdProvider.future);
15   if (id == null) {
16     yield const ProfilePrefs();
17     return;
18   }
19   yield* ref.watch(preferencesRepositoryProvider).watch(id);
20 });
```

ماذا يفعل؟ `activePrefsProvider`: بث بتفضيلات الملف النشط. ننتظر معرّف الملف النشط؛ إن لم يوجد نبثّ الافتراضيات ونتوقف، وإلا نمرّر بثّ المستودع كما هو.

لماذا بهذه الطريقة؟ `async*` و`yield*` يسمحان بتأجيل البث حتى يصل المعرّف. نراقب (`watch`) معرّف الملف فإذا بدّل الطالب ملفه انتقل البث إلى تفضيلات الملف الجديد تلقائيًا، فيتبدّل حجم الخط معه.

الأسطر 22–25

```dart
/// عدد النتائج المحفوظة على الجهاز وتنتظر الإرسال (طابور المزامنة).
final pendingSyncCountProvider = StreamProvider.autoDispose<int>((ref) {
  return ref.watch(appDatabaseProvider).activityDao.watchPendingSyncCount();
});
```

ماذا يفعل؟ `pendingSyncCountProvider`: بث بعدد صفوف طابور المزامنة.

لماذا بهذه الطريقة؟ `StreamProvider` فيتحدّث الرقم فور إضافة نتيجة جديدة بلا إعادة فتح الشاشة. `autoDispose` ينظّفه عند مغادرتها.

الأسطر 27–28

```dart
/// يشارك نصًّا عبر نافذة المشاركة في النظام.
typedef ShareText = Future<void> Function(String text);
```

ماذا يفعل؟ `typedef ShareText`: دالة تأخذ نصًّا وتعيد `Future<void>`.

لماذا بهذه الطريقة؟ نخفي حزمة المشاركة خلف توقيع بسيط: الشاشة لا تعرف `SharePlus` أصلًا.

الأسطر 30–35

```dart
/// منفصل كمزوّد ليُستبدل في الاختبارات (نافذة النظام لا تعمل فيها).
final shareTextProvider = Provider<ShareText>((ref) {
  return (text) async {
    await SharePlus.instance.share(ShareParams(text: text));
  };
});
```

ماذا يفعل؟ `shareTextProvider`: يعيد دالة تستدعي `SharePlus.instance.share(ShareParams(text: ...))`.

لماذا بهذه الطريقة؟ مزوّد منفصل ليُستبدل في الاختبارات (نافذة النظام لا تعمل فيها) ونتحقق أن النص المرسل صحيح.

lib/features/account/pin_manage_sheet.dart — ورقة تغيير الرمز السري أو إلغائه

lib/features/account/pin_manage_sheet.dart

الأسطر 1–6

```dart
import 'package:flutter/material.dart';

import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
```

ماذا يفعل؟ استيراد Material والنصوص والألوان والأنماط ومربّع الأيقونة.

لماذا بهذه الطريقة؟ ما تحتاجه الورقة.

السطر 8

```
8   enum PinManageAction { change, disable }
```

ماذا يفعل؟ تعداد `PinManageAction { change, disable }`.

لماذا بهذه الطريقة؟ نعيد اختيار الطالب كقيمة مسمّاة لا كنص أو رقم، فيفحصها `switch` شاملة في الشاشة.

الأسطر 10–12

```
10  /// ورقة سفلية لمن لديه رمز سري مفعّل: تغييره أو إلغاؤه.
11  /// تعيد null إن أغلقها الطالب دون اختيار.
12  Future<PinManageAction?> showPinManageSheet(BuildContext context) {
```

ماذا يفعل؟ تعليق ودالة `showPinManageSheet` تعيد `Future<PinManageAction?>`.

لماذا بهذه الطريقة؟ `null` تعني أن الطالب أغلق الورقة دون اختيار؛ الشاشة تتجاهلها.

الأسطر 13–18

```
13    return showModalBottomSheet<PinManageAction>(
14      context: context,
15      backgroundColor: AppColors.ground,
16      shape: const RoundedRectangleBorder(
17        borderRadius: BorderRadius.vertical(top: Radius.circular(28)),
18      ),
```

ماذا يفعل؟ `showModalBottomSheet` بخلفية `ground` وزوايا علوية مدوّرة 28.

لماذا بهذه الطريقة؟ نفس شكل ورقتي الرمز السري و«نسيت الرمز؟» فتبدو كلها من عائلة واحدة.

الأسطر 19–21

```
19      builder: (context) => SafeArea(
20        child: Padding(
21          padding: const EdgeInsetsDirectional.fromSTEB(20, 24, 20, 16),
```

ماذا يفعل؟ `SafeArea` ثم حشوة 20/24/20/16 (`Directional`).

لماذا بهذه الطريقة؟ تمنع تداخل الورقة مع حافة الجهاز السفلية.

الأسطر 22–26

```
22          child: Column(
23            mainAxisSize: MainAxisSize.min,
24            crossAxisAlignment: CrossAxisAlignment.stretch,
25            spacing: 8,
26            children: [
```

ماذا يفعل؟ `Column` بأقل حجم ممكن ومسافة 8 بين العناصر.

لماذا بهذه الطريقة؟ `mainAxisSize.min` فتأخذ الورقة طول محتواها فقط لا الشاشة كلها.

السطر 27

```
27              Text(ArStrings.pinManageTitle, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان الورقة.

لماذا بهذه الطريقة؟ بنمط `section18` كباقي العناوين.

الأسطر 28–34

```
28              _ActionRow(
29                icon: Icons.edit_outlined,
30                iconColor: AppColors.teal,
31                tint: AppColors.tealTint,
32                label: ArStrings.pinChange,
33                onTap: () => Navigator.of(context).pop(PinManageAction.change),
34              ),
```

ماذا يفعل؟ صف «تغيير الرمز»: أيقونة تعديل خضراء، وعند الضغط `pop(change)`.

لماذا بهذه الطريقة؟ `Navigator.pop` بقيمة تُعاد لمن فتح الورقة.

الأسطر 35–41

```
35              _ActionRow(
36                icon: Icons.lock_open_outlined,
37                iconColor: AppColors.coral,
38                tint: AppColors.coralTint,
39                label: ArStrings.pinDisable,
40                onTap: () => Navigator.of(context).pop(PinManageAction.disable),
41              ),
```

ماذا يفعل؟ صف «إلغاء الرمز السري»: أيقونة قفل مفتوح مرجانية، وعند الضغط `pop(disable)`.

لماذا بهذه الطريقة؟ اللون المرجاني يلمّح أنه إجراء يُضعف الحماية.

الأسطر 42–47

```
42            ],
43          ),
44        ),
45      ),
46    );
47  }
```

ماذا يفعل؟ إغلاق الأقواس.

لماذا بهذه الطريقة؟ نهاية الدالة.

الأسطر 49–56

```
class _ActionRow extends StatelessWidget {
  const _ActionRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.label,
    required this.onTap,
  });
```

ماذا يفعل؟ صنف خاص `_ActionRow` ومُنشئه بخمسة معاملات.

لماذا بهذه الطريقة؟ ودجت صغير خاص بهذا الملف لأن لا أحد غيره يحتاجه.

الأسطر 58–62

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String label;
  final VoidCallback onTap;
```

ماذا يفعل؟ الحقول: الأيقونة ولونها وخلفيتها والعنوان ودالة الضغط.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 64–66

```
  @override
  Widget build(BuildContext context) {
    return Material(
```

ماذا يفعل؟ `build` ويعيد `Material` ببطاقة بيضاء.

لماذا بهذه الطريقة؟ `Material` ضرورية ليظهر أثر اللمس (`InkWell`) فوق اللون.

الأسطر 67–69

```
      color: AppColors.card,
      borderRadius: BorderRadius.circular(18),
      clipBehavior: Clip.antiAlias,
```

ماذا يفعل؟ لون البطاقة وزوايا 18 و`clipBehavior` ليُقصّ أثر اللمس بالزوايا.

لماذا بهذه الطريقة؟ بدون القص يخرج الموج عن الزوايا المدوّرة.

الأسطر 70–74

```
      child: InkWell(
        onTap: onTap,
        child: Padding(
          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 10),
          child: Row(
```

ماذا يفعل؟ `InkWell` للضغط، وحشوة أفقية 14 ورأسية 10، و`Row` بمسافة 12.

لماذا بهذه الطريقة؟ أول عنصر في `Row` على اليمين (RTL): مربّع الأيقونة.

الأسطر 75–76

```
            spacing: 12,
            children: [
```

ماذا يفعل؟ `spacing: 12` و`children`.

لماذا بهذه الطريقة؟ مسافة ثابتة بدل `SizedBox` بين كل عنصرين.

الأسطر 77–84

```
              IconTile(
                iconData: icon,
                iconColor: iconColor,
                color: tint,
                size: 40,
                iconSize: 20,
                radius: 12,
              ),
```

ماذا يفعل؟ `IconTile` بحجم 40 وأيقونة 20 وزوايا 12.

لماذا بهذه الطريقة؟ نفس مقاسات صفوف الإعداد في التصميم.

السطر 85

```
              Expanded(child: Text(label, style: AppTextStyles.body15)),
```

ماذا يفعل؟ `Expanded` فيه نص العنوان.

لماذا بهذه الطريقة؟ ليأخذ العرض المتبقي ولا يتجاوز الحافة إن طال.

الأسطر 86–92

```
            ],
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق `Row` و`Padding` و`InkWell` و`Material` والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/account/account_screen.dart — الشاشة 19: حسابي

`lib/features/account/account_screen.dart`

الأسطر 1–3

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material وRiverpod وgo_router.

لماذا بهذه الطريقة؟ نستعمل `context.push/go` للتنقل.

الأسطر 5–6

```dart
import '../../app/routes.dart';
import '../../app/tab_navigation.dart';
```

ماذا يفعل؟ استيراد المسارات ودالة التنقل بين التبويبات.

لماذا بهذه الطريقة؟ الشاشة تعرف أسماء المسارات لا الراوتر.

الأسطر 7–14

```dart
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/theme/profile_palette.dart';
import '../../core/widgets/bottom_nav.dart';
import '../../core/widgets/mascot.dart';
import '../../core/widgets/rizma_card.dart';
```

ماذا يفعل؟ استيراد أنواع القاعدة والنصوص والألوان والأنماط وألوان الشخصية والشريط السفلي والشخصية والبطاقة.

لماذا بهذه الطريقة؟ `app_database` لنوع `Profile`. مكوّنات مشتركة موجودة سلفًا.

الأسطر 15–18

```dart
import '../catalog/catalog_providers.dart';
import '../profiles/active_profile_provider.dart';
import '../profiles/pin_setup_sheet.dart';
import '../profiles/profile_providers.dart';
```

ماذا يفعل؟ استيراد الكتالوج والملف النشط وورقة إعداد الرمز السري ومزوّدات الملفات.

لماذا بهذه الطريقة؟ الكتالوج لمعرفة صف المحتوى المتوفّر عند تغيير الصف. ورقة إعداد الرمز بُنيت للمرحلة 3 ونعيد استعمالها.

الأسطر 19–23

```dart
import 'account_providers.dart';
import 'pin_manage_sheet.dart';
import 'preferences_repository.dart';
import 'widgets/account_sheets.dart';
import 'widgets/setting_widgets.dart';
```

ماذا يفعل؟ استيراد مزوّدات حسابي وورقة إدارة الرمز ومستودع التفضيلات والأوراق وودجتات الإعدادات.

لماذا بهذه الطريقة؟ كل قطعة في ملفها؛ الشاشة تجمعها فقط.

الأسطر 25–28

```
25  /// الشاشة 19 «حسابي»: الملف، الدراسة، التنزيل والتخزين، عام.
26  /// الوضع الليلي» معطّل («قريبًا») لأن الثيم الداكن لم يُبنَ بعد، ومفاتيح«
27  /// التخزين والأصوات تُحفظ الآن وتُقرأ حين تُبنى ميزاتها.
28  class AccountScreen extends ConsumerWidget {
```

ماذا يفعل؟ تعليق وصنف `AccountScreen` من نوع `ConsumerWidget`.

لماذا بهذه الطريقة؟ التعليق يوثّق قرارين: «الوضع الليلي» معطّل «قريبًا» لأن الثيم الداكن لم يُبنَ، ومفاتيح التخزين والأصوات تُحفظ الآن وتُقرأ حين تُبنى ميزاتها. لا حالة محلية: كل شيء من مزوّدات.

السطر 29

```
29    const AccountScreen({super.key});
```

ماذا يفعل؟ باني ثابت.

لماذا بهذه الطريقة؟ `const` لأن الشاشة بلا حقول.

الأسطر 31–35

```
31    void _note(BuildContext context, String message) {
32      ScaffoldMessenger.of(
33        context,
34      ).showSnackBar(SnackBar(content: Text(message)));
35    }
```

ماذا يفعل؟ `_note`: يعرض `SnackBar` بنص.

لماذا بهذه الطريقة؟ دالة واحدة لكل رسائل التأكيد (رمز سري، صف).

الأسطر 37–44

```
37    Future<void> _onPinTap(
38      BuildContext context,
39      WidgetRef ref,
40      Profile profile,
41    ) async {
42      final repo = ref.read(profileRepositoryProvider);
43      final enabled = profile.pinHash != null;
```

ماذا يفعل؟ `_onPinTap(context, ref, profile)`: دالة غير متزامنة تُدير ضغط صف الرمز السري، ونقرأ مستودع الملفات ونعرف هل الرمز مفعّل.

لماذا بهذه الطريقة؟ `read` لا `watch` لأننا في معالج ضغط. الحالة من `pinHash != null` لا من متغيّر منفصل قد يختلف عنها.

الأسطر 45–51

```
45      if (!enabled) {
46        final pin = await showPinSetupSheet(context);
47        if (pin == null) return;
48        await repo.setPin(profile.id, pin);
49        if (context.mounted) _note(context, ArStrings.pinEnabledNote);
50        return;
51      }
```

ماذا يفعل؟ إن لم يكن مفعّلًا: نفتح ورقة إعداد الرمز؛ إن أغلقها الطالب (`null`) نخرج، وإلا نحفظه ونعرض «تم تفعيل الرمز السري».

لماذا بهذه الطريقة؟ الرمز يُجزّأ داخل `setPin` (SHA-256 مع salt) فلا يصل نصًّا إلى القاعدة. `context.mounted` تحمي من استعمال `context` بعد انتهاء الشاشة أثناء الانتظار.

الأسطر 53–54

```
53      final action = await showPinManageSheet(context);
54      if (action == null || !context.mounted) return;
```

ماذا يفعل؟ إن كان مفعّلًا: نفتح ورقة الإدارة؛ `null` أو شاشة مغلقة ← نخرج.

لماذا بهذه الطريقة؟ نتحقق من `mounted` مرة ثانية لأن فتح ورقة سفلية عملية انتظار.

الأسطر 55–60

```
55      switch (action) {
56        case PinManageAction.change:
57          final pin = await showPinSetupSheet(context);
58          if (pin == null) return;
59          await repo.setPin(profile.id, pin);
60          if (context.mounted) _note(context, ArStrings.pinChangedNote);
```

ماذا يفعل؟ `switch` على الإجراء: «تغيير» يفتح ورقة الإعداد ويحفظ الرمز الجديد ويعرض رسالة.

لماذا بهذه الطريقة؟ نفس خطوات التفعيل؛ `setPin` يولّد salt جديدًا فتتغيّر التجزئة كليًا.

الأسطر 61–65

```
61        case PinManageAction.disable:
62          await repo.clearPin(profile.id);
63          if (context.mounted) _note(context, ArStrings.pinDisabledNote);
64      }
65    }
```

ماذا يفعل؟ «إلغاء» ينادي `clearPin` ويعرض «أُلغي الرمز السري»، ثم إغلاق `switch` والدالة.

لماذا بهذه الطريقة؟ `clearPin` يمسح التجزئة والـ salt معًا ويصفّر عدّاد القفل.

الأسطر 67–71

```
67    Future<void> _onGradeTap(
68      BuildContext context,
69      WidgetRef ref,
70      Profile profile,
71    ) async {
```

ماذا يفعل؟ `_onGradeTap`: دالة غير متزامنة بمعاملات السياق و`ref` والملف.

لماذا بهذه الطريقة؟ تعالج ضغط صف «الصف».

الأسطر 72–78

```
72      // صف الكتالوج المتوفّر؛ إن اختلف عن المختار نخبر الطالب.
73      final contentGrade = ref.read(catalogProvider).value?.grade;
74      final grade = await showGradeSheet(
75        context,
76        current: profile.grade,
77        contentGrade: contentGrade,
78      );
```

ماذا يفعل؟ نقرأ صف الكتالوج المتوفّر (`catalogProvider` قد لا يكون جاهزًا فيكون `null`) ونفتح ورقة اختيار الصف.

لماذا بهذه الطريقة؟ `ref.read(...).value?.grade`: لو لم يُحمَّل الكتالوج بعد نكتفي بعدم عرض تنبيه المحتوى بدل أن نمنع التغيير.

السطر 79

```
79      if (grade == null || grade == profile.grade) return;
```

ماذا يفعل؟ إن أغلق الطالب الورقة أو اختار صفّه الحالي نخرج.

لماذا بهذه الطريقة؟ لا كتابة في القاعدة بلا تغيير حقيقي.

الأسطر 80–82

```
80      await ref.read(profileRepositoryProvider).setGrade(profile.id, grade);
81      if (context.mounted) _note(context, ArStrings.gradeChanged(grade));
82    }
```

ماذا يفعل؟ نحدّث الصف عبر المستودع ونعرض «تم تغيير الصف إلى ...».

لماذا بهذه الطريقة؟ التحقق من النطاق في المستودع. الشاشة تتحدّث وحدها لأن `activeProfileProvider` بث من القاعدة.

الأسطر 84–89

```
84    Future<void> _onFontTap(
85      BuildContext context,
86      WidgetRef ref,
87      Profile profile,
88      ProfilePrefs prefs,
89    ) async {
```

ماذا يفعل؟ `_onFontTap`: دالة غير متزامنة بمعاملات السياق و`ref` والملف والتفضيلات الحالية.

لماذا بهذه الطريقة؟ التفضيلات الحالية لتعليم الخيار المختار.

الأسطر 90–95

```
90      final choice = await showChoiceSheet<FontSizeChoice>(
91        context,
92        title: ArStrings.fontSheetTitle,
93        options: [for (final c in FontSizeChoice.values) (c, _fontSizeLabel(c))],
94        selected: prefs.fontSize,
95      );
```

ماذا يفعل؟ نفتح ورقة الاختيار من القيم الثلاث لـ `FontSizeChoice` مع اسم كل قيمة، ونعلّم الحالية.

لماذا بهذه الطريقة؟ القائمة تُبنى من التعداد نفسه فلا تختلف قيمة في الورقة عن قيمة في القاعدة. `(c, label)` سجل Dart 3.

الأسطر 96–100

```
96      if (choice == null) return;
97      await ref
98          .read(preferencesRepositoryProvider)
99          .setFontSize(profile.id, choice);
100   }
```

ماذا يفعل؟ إن اختار الطالب قيمة نحفظها لهذا الملف.

لماذا بهذه الطريقة؟ لا نطبّق الحجم يدويًا: `RizmaApp` يراقب التفضيلات فيتغيّر الخط في كل الشاشات فورًا.

الأسطر 102–107

```
102   @override
103   Widget build(BuildContext context, WidgetRef ref) {
104     final profile = ref.watch(activeProfileProvider);
105     final stars = ref.watch(starsByProfileProvider).value ?? const {};
106     final prefs = ref.watch(activePrefsProvider).value ?? const ProfilePrefs();
107     final prefsRepo = ref.read(preferencesRepositoryProvider);
```

ماذا يفعل؟ `build`: نراقب الملف النشط ونجوم كل الملفات والتفضيلات (افتراضيات إلى أن تصل)، ونقرأ مستودع التفضيلات.

لماذا بهذه الطريقة؟ `.value ?? const {}` و`?? const ProfilePrefs()`: إن لم تصل القيمة بعد نعرض افتراضيًا بدل خطأ. النجوم محسوبة من المحاولات ولا تُخزَّن.

الأسطر 109–112

```
109     return Scaffold(
110       body: SafeArea(
111         bottom: false,
112         child: profile.when(
```

ماذا يفعل؟ `Scaffold` و`SafeArea` بلا حماية من الأسفل ثم `profile.when`.

لماذا بهذه الطريقة؟ الشريط السفلي يعالج الأسفل بنفسه. `when` تفرّق التحميل والخطأ والبيانات.

الأسطر 113–119

```
          loading: () => const SizedBox.shrink(),
          error: (_, _) => Center(
            child: Text(
              ArStrings.accountLoadFailed,
              style: AppTextStyles.body15,
            ),
          ),
```

ماذا يفعل؟ التحميل: فراغ. الخطأ: «تعذّر تحميل حسابك.» في المنتصف.

لماذا بهذه الطريقة؟ رسالة واضحة بدل شاشة بيضاء.

الأسطر 120–123

```
          data: (p) => p == null
              ? const SizedBox.shrink()
              : SingleChildScrollView(
                  padding: const EdgeInsetsDirectional.fromSTEB(20, 18, 20, 24),
```

ماذا يفعل؟ البيانات: إن كان `null` (لا ملف نشط) فراغ، وإلا `SingleChildScrollView` بحشوة 20/18/20/24.

لماذا بهذه الطريقة؟ الراوتر يحوّل للإقلاع في تلك الحالة؛ الفراغ احتياط لحظة التحويل. الصفحة طويلة الآن (خمس بطاقات) فالتمرير ضروري.

الأسطر 124–127

```
                child: Column(
                  crossAxisAlignment: CrossAxisAlignment.stretch,
                  spacing: 14,
                  children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 14.

لماذا بهذه الطريقة؟ تباعد البطاقات.

الأسطر 128–131

```
                    Text(
                      ArStrings.accountTitle,
                      style: AppTextStyles.heading22,
                    ),
```

ماذا يفعل؟ عنوان «حسابي» بنمط `heading22`.

لماذا بهذه الطريقة؟ في أعلى الشاشة كما في التصميم.

السطر 132

```
                    _ProfileHeader(profile: p, stars: stars[p.id] ?? 0),
```

ماذا يفعل؟ رأس الملف: `_ProfileHeader` بالملف وعدد نجومه (`?? 0`).

لماذا بهذه الطريقة؟ ملف بلا محاولات غير موجود في الخريطة فنعرض 0.

الأسطر 133–139

```
                    _ProfileGroup(
                      profile: p,
                      onStudentCode: () =>
                          context.push(AppRoutes.studentCode),
                      onSwitch: () => context.go(AppRoutes.profiles),
                      onPin: () => _onPinTap(context, ref, p),
                    ),
```

ماذا يفعل؟ `_ProfileGroup` بثلاث دوال: رمز الطالب (`push`)، تبديل الطالب (`go` إلى `/profiles`)، والرمز السري.

لماذا بهذه الطريقة؟ `push` لرمز الطالب ليعود زر الرجوع إلى حسابي. `go` لتبديل الطالب لأنها بداية جلسة طالب آخر لا شاشة فرعية.

الأسطر 140–146

```
                    _StudyGroup(
                      profile: p,
                      prefs: prefs,
                      onGrade: () => _onGradeTap(context, ref, p),
                      onFontSize: () => _onFontTap(context, ref, p, prefs),
                      onGameSounds: (v) => prefsRepo.setGameSounds(p.id, v),
                    ),
```

ماذا يفعل؟ `_StudyGroup` بالملف والتفضيلات ودوال الصف وحجم الخط وأصوات الألعاب (تحفظ القيمة مباشرة).

لماذا بهذه الطريقة؟ دالة المفتاح تأخذ القيمة الجديدة `v` فتحفظها: المفتاح لا يحتفظ بحالة، الحالة في القاعدة.

الأسطر 147–152

```
                    _StorageGroup(
                      prefs: prefs,
                      onWifiOnly: (v) => prefsRepo.setWifiOnly(p.id, v),
                      onAutoVideo: (v) => prefsRepo.setAutoVideo(p.id, v),
                      onManage: () => context.push(AppRoutes.downloads),
                    ),
```

ماذا يفعل؟ `_StorageGroup` بالتفضيلات ودوال Wi-Fi وتنزيل الفيديو وإدارة التنزيلات (`push`).

لماذا بهذه الطريقة؟ إدارة التنزيلات شاشة فرعية.

الأسطر 153–172

```
                    _GeneralGroup(
                      onPrivacy: () => showInfoSheet(
                        context,
                        title: ArStrings.privacyTitle,
                        paragraphs: const [
                          ArStrings.privacy1,
                          ArStrings.privacy2,
                          ArStrings.privacy3,
                        ],
                      ),
                      onAbout: () => showInfoSheet(
                        context,
                        title: ArStrings.aboutTitle,
                        paragraphs: const [
                          ArStrings.about1,
                          ArStrings.about2,
                          ArStrings.about3,
                        ],
                      ),
                    ),
```

ماذا يفعل؟ `_GeneralGroup` بدالتين تفتحان ورقة معلومات: الخصوصية (ثلاث فقرات) وعن رزمة (ثلاث فقرات).

لماذا بهذه الطريقة؟ النصوص من `ArStrings` بقوائم `const`. لا شاشتان كاملتان لأن المحتوى نصّ قصير ثابت.

الأسطر 173–177

```
                  ],
                ),
              ),
        ),
      ),
```

ماذا يفعل؟ إغلاق العناصر والـ `Column` و`SingleChildScrollView` و`when` و`SafeArea`.

لماذا بهذه الطريقة؟ نهاية الجسم.

الأسطر 178–181

```
      bottomNavigationBar: BottomNav(
        current: RizmaTab.profile,
        onSelected: (tab) => onTabSelected(context, RizmaTab.profile, tab),
      ),
```

ماذا يفعل؟ الشريط السفلي بالتبويب «حسابي»، والاختيار يمرّ بـ `onTabSelected`.

لماذا بهذه الطريقة؟ نفس منطق التنقل في كل الشاشات.

الأسطر 182–184

```
    );
  }
}
```

ماذا يفعل؟ إغلاق `Scaffold` و`build` والصنف.

لماذا بهذه الطريقة؟ نهاية `AccountScreen`.

الأسطر 186–190

```
String _fontSizeLabel(FontSizeChoice choice) => switch (choice) {
  FontSizeChoice.small => ArStrings.fontSizeSmall,
  FontSizeChoice.medium => ArStrings.fontSizeMedium,
  FontSizeChoice.large => ArStrings.fontSizeLarge,
};
```

ماذا يفعل؟ دالة `_fontSizeLabel`: اسم عربي لكل قيمة من `FontSizeChoice` بتعبير `switch` شامل.

لماذا بهذه الطريقة؟ خارج الصنف لأنها تُستعمل في الشاشة وفي مجموعة الدراسة. `switch` شاملة على التعداد: لو أضفنا قيمة رابعة نسيناها يحذّرنا المحلّل.

الأسطر 192–196

```
class _ProfileHeader extends StatelessWidget {
  const _ProfileHeader({required this.profile, required this.stars});

  final Profile profile;
  final int stars;
```

ماذا يفعل؟ ودجت `_ProfileHeader` بمعاملين: الملف وعدد النجوم.

لماذا بهذه الطريقة؟ يفصل رأس الملف عن بقية الشاشة.

الأسطر 198–204

```
  @override
  Widget build(BuildContext context) {
    return RizmaCard(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
      child: Column(
        spacing: 4,
        children: [
```

ماذا يفعل؟ `build` ويعيد `RizmaCard` بحشوة 16/20 و`Column` بمسافة 4.

لماذا بهذه الطريقة؟ بطاقة بيضاء بحدّ رفيع كما في التصميم.

الأسطر 205–208

```
205            Container(
206              width: 100,
207              height: 100,
208              margin: const EdgeInsets.only(bottom: 8),
```

ماذا يفعل؟ حاوية 100×100 بهامش سفلي 8.

لماذا بهذه الطريقة؟ الأفاتار الكبير في التصميم 100.

الأسطر 209–215

```
209              decoration: BoxDecoration(
210                color: profile.color.avatarBackground,
211                shape: BoxShape.circle,
212              ),
213              alignment: Alignment.center,
214              child: Mascot(size: 76, color: profile.color),
215            ),
```

ماذا يفعل؟ خلفية بلون الأفاتار الخاص بلون الطالب وشكل دائري، والشخصية بحجم 76 بلون الطالب.

لماذا بهذه الطريقة؟ `Mascot` تلوّن الـ SVG وقت العرض فلا نحتاج خمسة ملفات. تتبدّل بتبدّل لون الطالب.

السطر 216

```
216            Text(profile.name, style: AppTextStyles.title26),
```

ماذا يفعل؟ اسم الطالب بنمط `title26`.

لماذا بهذه الطريقة؟ الاسم من الملف لا نص ثابت.

الأسطر 217–220

```
217            Text(
218              ArStrings.accountSummary(profile.grade, stars),
219              style: AppTextStyles.label14,
220            ),
```

ماذا يفعل؟ سطر «الصف الخامس · 120 نجمة» عبر `accountSummary`.

لماذا بهذه الطريقة؟ دالة نصوص تعيد استعمال صيغة الجمع العربية للنجوم. يتغيّر فور تغيير الصف.

الأسطر 221–225

```
221          ],
222        ),
223      );
224    }
225  }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_ProfileHeader`.

الأسطر 227–238

```
class _ProfileGroup extends StatelessWidget {
  const _ProfileGroup({
    required this.profile,
    required this.onStudentCode,
    required this.onSwitch,
    required this.onPin,
  });

  final Profile profile;
  final VoidCallback onStudentCode;
  final VoidCallback onSwitch;
  final VoidCallback onPin;
```

ماذا يفعل؟ ودجت `_ProfileGroup` ومعاملاته وحقوله: الملف وثلاث دوال ضغط.

لماذا بهذه الطريقة؟ الودجت لا يعرف التنقل؛ يستقبله دوالّ جاهزة (قابل للاختبار).

الأسطر 240–246

```
  @override
  Widget build(BuildContext context) {
    final pinOn = profile.pinHash != null;

    return SettingsGroupCard(
      title: ArStrings.accountProfileGroup,
      children: [
```

ماذا يفعل؟ `build` ونحسب `pinOn = pinHash != null` ثم `SettingsGroupCard` بعنوان «الملف».

لماذا بهذه الطريقة؟ مصدر الحقيقة الوحيد لحالة الرمز. بطاقة المجموعة مشتركة بين المجموعات الأربع.

الأسطر 247–256

```
        SettingRow(
          icon: Icons.vpn_key_outlined,
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountStudentCode,
          value: profile.studentCode,
          valueColor: AppColors.teal,
          valueLtr: true,
          onTap: onStudentCode,
        ),
```

ماذا يفعل؟ صف «رمز الطالب»: أيقونة مفتاح خضراء، القيمة هي الرمز نفسه (LTR) بلون أخضر، وعند الضغط يفتح شاشة 29.

لماذا بهذه الطريقة؟ `valueLtr` لأن `RZ-7K4M` يجب أن يُقرأ من اليسار حتى داخل واجهة RTL.

الأسطر 257–263

```
        SettingRow(
          icon: Icons.group_outlined,
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountSwitch,
          onTap: onSwitch,
        ),
```

ماذا يفعل؟ صف «تبديل الطالب»: أيقونة مستخدمين زرقاء بلا قيمة.

لماذا بهذه الطريقة؟ يفتح شاشة اختيار الملف.

الأسطر 264–272

```
        SettingRow(
          icon: Icons.lock_outline,
          iconColor: AppColors.purple,
          tint: AppColors.purpleTint,
          label: ArStrings.accountPin,
          value: pinOn ? ArStrings.accountPinOn : ArStrings.accountPinOff,
          valueColor: pinOn ? AppColors.teal : AppColors.soft,
          onTap: onPin,
        ),
```

ماذا يفعل؟ صف «الرمز السري»: أيقونة قفل بنفسجية، والقيمة «مفعّل» (أخضر) أو «غير مفعّل» (رمادي).

لماذا بهذه الطريقة؟ اللون يعطي إشارة سريعة للحالة.

الأسطر 273–276

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق قائمة الصفوف والمجموعة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `_ProfileGroup`.

الأسطر 278–291

```
class _StudyGroup extends StatelessWidget {
  const _StudyGroup({
    required this.profile,
    required this.prefs,
    required this.onGrade,
    required this.onFontSize,
    required this.onGameSounds,
  });

  final Profile profile;
  final ProfilePrefs prefs;
  final VoidCallback onGrade;
  final VoidCallback onFontSize;
  final ValueChanged<bool> onGameSounds;
```

ماذا يفعل؟ ودجت `_StudyGroup` ومعاملاته وحقوله: الملف والتفضيلات ودوال الصف والخط والأصوات.

لماذا بهذه الطريقة؟ يستقبل `ValueChanged<bool>` للمفتاح لأنه يحتاج القيمة الجديدة.

الأسطر 322–325

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق قائمة الصفوف والمجموعة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `_StudyGroup`.

الأسطر 293–297

```
  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountStudyGroup,
      children: [
```

ماذا يفعل؟ `build` ويعيد `SettingsGroupCard` بعنوان «الدراسة».

لماذا بهذه الطريقة؟ نفس الشكل.

الأسطر 298–305

```
        SettingRow(
          icon: Icons.menu_book_outlined,
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountGrade,
          value: ArStrings.gradeOrdinal(profile.grade),
          onTap: onGrade,
        ),
```

ماذا يفعل؟ صف «الصف»: أيقونة كتاب، والقيمة اسم الصف الترتيبي («الخامس»)، وعند الضغط ورقة اختيار الصف.

لماذا بهذه الطريقة؟ `gradeOrdinal` تعطي «الخامس» لا «الصف الخامس» لأن العنوان «الصف» بجانبه.

الأسطر 306–313

```
        SettingRow(
          icon: Icons.text_fields_rounded,
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountFontSize,
          value: _fontSizeLabel(prefs.fontSize),
          onTap: onFontSize,
        ),
```

ماذا يفعل؟ صف «حجم الخط»: أيقونة نص، والقيمة «صغير/متوسط/كبير»، وعند الضغط ورقة الاختيار.

لماذا بهذه الطريقة؟ القيمة من التفضيلات المحفوظة.

الأسطر 314–321

```
        SettingSwitchRow(
          icon: Icons.volume_up_outlined,
          iconColor: AppColors.purple,
          tint: AppColors.purpleTint,
          label: ArStrings.accountGameSounds,
          value: prefs.gameSounds,
          onChanged: onGameSounds,
        ),
```

ماذا يفعل؟ صف «أصوات الألعاب» بمفتاح.

لماذا بهذه الطريقة؟ مفتاح يحفظ الإعداد الآن؛ لا أثر له قبل بناء الألعاب (مذكور في الشرح الموضوعي).

الأسطر 327–338

```
class _StorageGroup extends StatelessWidget {
  const _StorageGroup({
    required this.prefs,
    required this.onWifiOnly,
    required this.onAutoVideo,
    required this.onManage,
  });

  final ProfilePrefs prefs;
  final ValueChanged<bool> onWifiOnly;
  final ValueChanged<bool> onAutoVideo;
  final VoidCallback onManage;
```

ماذا يفعل؟ ودجت `_StorageGroup` ومعاملاته وحقوله.

لماذا بهذه الطريقة؟ التفضيلات ودالتا المفتاحين وإدارة التنزيلات.

الأسطر 340–345

```
  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountStorageGroup,
      footnote: ArStrings.accountStorageNote,
      children: [
```

ماذا يفعل؟ `build` ويعيد `SettingsGroupCard` بعنوان «التنزيل والتخزين» وملاحظة سفلية.

لماذا بهذه الطريقة؟ الملاحظة «يعمل مع التنزيلات عند توفرها» صادقة: المفاتيح بلا أثر قبل مدير التنزيل.

الأسطر 346–361

```
        SettingSwitchRow(
          icon: Icons.wifi_rounded,
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountWifiOnly,
          value: prefs.wifiOnly,
          onChanged: onWifiOnly,
        ),
        SettingSwitchRow(
          icon: Icons.videocam_outlined,
          iconColor: AppColors.coral,
          tint: AppColors.coralTint,
          label: ArStrings.accountAutoVideo,
          value: prefs.autoVideo,
          onChanged: onAutoVideo,
        ),
```

ماذا يفعل؟ صفّا «التنزيل عبر Wi-Fi فقط» (مفتاح، افتراضيًا مفعّل) و«تنزيل الفيديو تلقائيًا» (مفتاح، افتراضيًا مطفأ).

لماذا بهذه الطريقة؟ الافتراضيان محافظان: لا نستهلك باقة الطالب دون إذنه.

الأسطر 362–368

```
        SettingRow(
          icon: Icons.download_outlined,
          iconColor: AppColors.blue,
          tint: AppColors.blueTint,
          label: ArStrings.accountManageDownloads,
          onTap: onManage,
        ),
```

ماذا يفعل؟ صف «إدارة التنزيلات» بسهم.

لماذا بهذه الطريقة؟ يفتح الحالة الفارغة الصادقة.

الأسطر 369–372

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق قائمة الصفوف والمجموعة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية `_StorageGroup` .

الأسطر 374–378

```
class _GeneralGroup extends StatelessWidget {
  const _GeneralGroup({required this.onPrivacy, required this.onAbout});

  final VoidCallback onPrivacy;
  final VoidCallback onAbout;
```

ماذا يفعل؟ ودجت `_GeneralGroup` ودالتا الخصوصية وعن رزمة.

لماذا بهذه الطريقة؟ يستقبل الدوال جاهزة.

الأسطر 380–384

```
  @override
  Widget build(BuildContext context) {
    return SettingsGroupCard(
      title: ArStrings.accountGeneralGroup,
      children: [
```

ماذا يفعل؟ `build` ويعيد `SettingsGroupCard` بعنوان «عام».

لماذا بهذه الطريقة؟ نفس الشكل.

الأسطر 385–390

```
        const SoonSettingRow(
          icon: Icons.nights_stay_outlined,
          iconColor: AppColors.soft,
          tint: AppColors.mute,
          label: ArStrings.accountNightMode,
        ),
```

ماذا يفعل؟ أول صف: `SoonSettingRow` للوضع الليلي.

لماذا بهذه الطريقة؟ `const` لأنه معطّل وبلا أي دالة: لا مفتاح حقيقي ولا حالة. الثيم الداكن غير مبني ولا نخدع الطالب بمفتاح لا يفعل شيئًا.

الأسطر 391–397

```
        SettingRow(
          icon: Icons.shield_outlined,
          iconColor: AppColors.teal,
          tint: AppColors.tealTint,
          label: ArStrings.accountPrivacy,
          onTap: onPrivacy,
        ),
```

ماذا يفعل؟ صف «الخصوصية: بياناتك تبقى على جهازك» يفتح ورقة معلومات.

لماذا بهذه الطريقة؟ جملة العنوان من التصميم وهي صحيحة الآن.

الأسطر 398–404

```
        SettingRow(
          icon: Icons.public_rounded,
          iconColor: AppColors.amberDark,
          tint: AppColors.amberTint,
          label: ArStrings.accountAbout,
          onTap: onAbout,
        ),
```

ماذا يفعل؟ صف «عن رزمة · مبادرة شبابية مجتمعية» يفتح ورقة معلومات.

لماذا بهذه الطريقة؟ من التصميم.

الأسطر 405–408

```
      ],
    );
  }
}
```

ماذا يفعل؟ إغلاق قائمة الصفوف والمجموعة والدالة والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.

lib/features/account/student_code_screen.dart — الشاشة 29: رمز الطالب

`lib/features/account/student_code_screen.dart`

الأسطر 1–4

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:go_router/go_router.dart';
```

ماذا يفعل؟ استيراد Material و`services` (للحافظة) وRiverpod وgo_router.

لماذا بهذه الطريقة؟ `Clipboard` تأتي من `services`.

الأسطر 6–15

```dart
import '../../app/routes.dart';
import '../../core/db/app_database.dart';
import '../../core/strings/ar.dart';
import '../../core/theme/app_colors.dart';
import '../../core/theme/app_text_styles.dart';
import '../../core/widgets/icon_tile.dart';
import '../../core/widgets/rizma_card.dart';
import '../../core/widgets/screen_top_bar.dart';
import '../profiles/active_profile_provider.dart';
import 'account_providers.dart';
```

ماذا يفعل؟ استيراد المسارات والقاعدة والنصوص والألوان والأنماط والمكوّنات والملف النشط ومزوّدات الحساب.

لماذا بهذه الطريقة؟ كلها موجودة سلفًا أو أُنشئت في هذه المرحلة.

الأسطر 17–20

```dart
/// الشاشة 29 «رمز الطالب»: الرمز مع نسخ ومشاركة، وسبب وجوده، وحالة الحفظ على
/// الإنترنت. مجموعة الصف تظهر فقط إن أدخل الطالب رمز صف.
class StudentCodeScreen extends ConsumerWidget {
  const StudentCodeScreen({super.key});
```

ماذا يفعل؟ تعليق وصنف `StudentCodeScreen` من نوع `ConsumerWidget`.

لماذا بهذه الطريقة؟ التعليق يذكر أن مجموعة الصف مشروطة بوجود رمز صف.

الأسطر 22–24

```dart
  void _back(BuildContext context) {
    context.canPop() ? context.pop() : context.go(AppRoutes.account);
  }
```

ماذا يفعل؟ `_back`: إن أمكن الرجوع (`canPop`) نرجع، وإلا نذهب إلى حسابي.

لماذا بهذه الطريقة؟ لو فُتحت الشاشة مباشرة (رابط) لا يوجد مكدس للرجوع إليه فلا نترك الطالب عالقًا.

الأسطر 26–32

```dart
  Future<void> _copy(BuildContext context, String code) async {
    await Clipboard.setData(ClipboardData(text: code));
    if (!context.mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(const SnackBar(content: Text(ArStrings.codeCopied)));
  }
```

ماذا يفعل؟ `_copy`: ننسخ الرمز للحافظة ثم نعرض «تم نسخ الرمز».

لماذا بهذه الطريقة؟ نتحقق من `mounted` بعد الانتظار قبل استعمال `context`. التأكيد البصري ضروري لأن النسخ لا يظهر شيئًا بنفسه.

الأسطر 34–37

```
34    @override
35    Widget build(BuildContext context, WidgetRef ref) {
36      final profile = ref.watch(activeProfileProvider).value;
37      final pending = ref.watch(pendingSyncCountProvider).value ?? 0;
```

ماذا يفعل؟ `build`: نراقب الملف النشط وعدد النتائج المنتظرة (`?? 0` إلى أن يصل).

لماذا بهذه الطريقة؟ عدد النتائج من طابور المزامنة الحقيقي.

الأسطر 39–43

```
39      return Scaffold(
40        body: SafeArea(
41          child: profile == null
42              ? const SizedBox.shrink()
43              : SingleChildScrollView(
```

ماذا يفعل؟ `Scaffold` و`SafeArea` ثم: بلا ملف نشط فراغ، وإلا `SingleChildScrollView`.

لماذا بهذه الطريقة؟ الشاشة تتمرّر على الشاشات القصيرة.

الأسطر 44–46

```
44                  child: Column(
45                    crossAxisAlignment: CrossAxisAlignment.stretch,
46                    children: [
```

ماذا يفعل؟ `Column` ممتد.

لماذا بهذه الطريقة؟ الشريط العلوي بعرض كامل.

الأسطر 47–51

```
47                      ScreenTopBar(
48                        title: ArStrings.codeTitle,
49                        subtitle: ArStrings.accountTitle,
50                        onBack: () => _back(context),
51                      ),
```

ماذا يفعل؟ `ScreenTopBar`: العنوان «رمز الطالب» والعنوان الفرعي «حسابي» وزر الرجوع.

لماذا بهذه الطريقة؟ مكوّن مشترك مع شاشات الإنشاء.

الأسطر 52–58

```
52                      Padding(
53                        padding: const EdgeInsetsDirectional.fromSTEB(
54                          20,
55                          12,
56                          20,
57                          24,
58                        ),
```

ماذا يفعل؟ حشوة 20/12/20/24 حول المحتوى.

لماذا بهذه الطريقة؟ هوامش التصميم.

الأسطر 59–62

```
59                      child: Column(
60                        crossAxisAlignment: CrossAxisAlignment.stretch,
61                        spacing: 14,
62                        children: [
```

ماذا يفعل؟ `Column` بمسافة 14 وممتد.

لماذا بهذه الطريقة؟ تباعد البطاقات.

الأسطر 63–69

```
63                          _CodeCard(
64                            code: profile.studentCode,
65                            onCopy: () => _copy(context, profile.studentCode),
66                            onShare: () => ref.read(shareTextProvider)(
67                              ArStrings.codeShareMessage(profile.studentCode),
68                            ),
69                          ),
```

ماذا يفعل؟ `_CodeCard` بالرمز ودالتي النسخ والمشاركة؛ المشاركة تستدعي `shareTextProvider` برسالة تحوي الرمز.

لماذا بهذه الطريقة؟ `ref.read` لأننا في معالج ضغط. المزوّد يُستبدل في الاختبار.

السطر 70

```
70                          const _WhyCard(),
```

ماذا يفعل؟ بطاقة «لماذا هذا الرمز؟» ثابتة.

لماذا بهذه الطريقة؟ `const` لأن لا بيانات فيها.

الأسطر 71–79

```
71                          _InfoRow(
72                            iconData: Icons.sync_rounded,
73                            iconColor: AppColors.teal,
74                            tint: AppColors.tealTint,
75                            title: ArStrings.codeNeverSynced,
76                            caption: pending == 0
77                                ? ArStrings.codeNothingPending
78                                : ArStrings.codePending(pending),
79                          ),
```

ماذا يفعل؟ بطاقة حالة الحفظ: العنوان «لم يُحفظ على الإنترنت بعد»، والسطر الثاني «N نتائج بانتظار الاتصال» أو «لا توجد نتائج».

لماذا بهذه الطريقة؟ صادقة: لا مزامنة بعد. وعند وجود نتائج نعرض عددها الحقيقي من الطابور.

الأسطر 80–81

```
80                          if (profile.classCode != null)
81                            _ClassGroupRow(profile: profile),
```

ماذا يفعل؟ بطاقة مجموعة الصف فقط إن كان `classCode != null` .

لماذا بهذه الطريقة؟ من لم يدخل رمز صف لا يرى مجموعة فارغة. `if` داخل القائمة.

السطر 82

```
                          const SizedBox(height: 4),
```

ماذا يفعل؟ مسافة 4 إضافية.

لماذا بهذه الطريقة؟ تفصل التذكير الأخير عن البطاقات.

الأسطر 83–87

```
                          Text(
                            ArStrings.codeKeepSafe,
                            textAlign: TextAlign.center,
                            style: AppTextStyles.caption13,
                          ),
```

ماذا يفعل؟ نص «احتفظ بالرمز في مكان آمن...» في الوسط بنمط `caption13`.

لماذا بهذه الطريقة؟ تذكير من التصميم.

الأسطر 88–97

```
                        ],
                      ),
                    ),
                  ],
                ),
              ),
            ),
          );
        }
      }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `StudentCodeScreen`.

الأسطر 99–108

```
class _CodeCard extends StatelessWidget {
  const _CodeCard({
    required this.code,
    required this.onCopy,
    required this.onShare,
  });

  final String code;
  final VoidCallback onCopy;
  final VoidCallback onShare;
```

ماذا يفعل؟ ودجت `_CodeCard` ومعاملاته: الرمز ودالتا النسخ والمشاركة.

لماذا بهذه الطريقة؟ يفصل البطاقة الداكنة.

الأسطر 110–117

```
@override
Widget build(BuildContext context) {
  return Container(
    padding: const EdgeInsets.all(20),
    decoration: BoxDecoration(
      color: AppColors.ink,
      borderRadius: BorderRadius.circular(24),
    ),
```

ماذا يفعل؟ `Container` في `build`: بحشوة 20 ولون `ink` وزوايا 24.

لماذا بهذه الطريقة؟ بطاقة داكنة كما في التصميم تجعل الرمز بارزًا.

الأسطر 118–120

```
    child: Column(
      spacing: 8,
      children: [
```

ماذا يفعل؟ `Column` بمسافة 8.

لماذا بهذه الطريقة؟ ترتيب: «رمزك» ثم الرمز ثم الأزرار.

الأسطر 121–126

```
        Text(
          ArStrings.codeYours,
          style: AppTextStyles.label14.copyWith(
            color: const Color(0xFFB9CFC8),
          ),
        ),
```

ماذا يفعل؟ نص «رمزك» بلون أخضر رمادي فاتح (`B9CFC8`).

لماذا بهذه الطريقة؟ لون التصميم لنص ثانوي على الداكن.

الأسطر 127–131

```
        Text(
          code,
          textDirection: TextDirection.ltr,
          style: AppTextStyles.wordmark40.copyWith(letterSpacing: 1),
        ),
```

ماذا يفعل؟ الرمز بحجم 40 وبـ `TextDirection.ltr` وتباعد حروف 1.

لماذا بهذه الطريقة؟ LTR كي لا ينقلب `RZ-7K4M`. `wordmark40` نمط موجود بحجم 40 ولون فاتح.

الأسطر 132–135

```
132            Padding(
133              padding: const EdgeInsets.only(top: 6),
134              child: Row(
135                spacing: 10,
```

ماذا يفعل؟ حشوة علوية 6 و`Row` بمسافة 10.

لماذا بهذه الطريقة؟ يفصل الأزرار عن الرمز.

الأسطر 136–143

```
136                children: [
137                  Expanded(
138                    child: _CodeButton(
139                      icon: Icons.copy_rounded,
140                      label: ArStrings.codeCopy,
141                      onTap: onCopy,
142                    ),
143                  ),
```

ماذا يفعل؟ زر «نسخ» داخل `Expanded`.

لماذا بهذه الطريقة؟ الزران بعرض متساوٍ.

الأسطر 144–150

```
144                  Expanded(
145                    child: _CodeButton(
146                      icon: Icons.share_outlined,
147                      label: ArStrings.codeShare,
148                      onTap: onShare,
149                    ),
150                  ),
```

ماذا يفعل؟ زر «مشاركة» داخل `Expanded`.

لماذا بهذه الطريقة؟ ترتيب RTL: «نسخ» يمين و«مشاركة» يسار كما في التصميم.

الأسطر 151–158

```
151                ],
152              ),
153            ),
154          ],
155        ),
156      );
157    }
158  }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_CodeCard`.

الأسطر 160–169

```
class _CodeButton extends StatelessWidget {
  const _CodeButton({
    required this.icon,
    required this.label,
    required this.onTap,
  });

  final IconData icon;
  final String label;
  final VoidCallback onTap;
```

ماذا يفعل؟ ودجت `_CodeButton` ومعاملاته: الأيقونة والعنوان والضغط.

لماذا بهذه الطريقة؟ زر صغير داكن خاص بهذه البطاقة.

الأسطر 171–176

```
  @override
  Widget build(BuildContext context) {
    return Semantics(
      button: true,
      label: label,
      excludeSemantics: true,
```

ماذا يفعل؟ `Semantics` : `build` زر بوصف العنوان.

لماذا بهذه الطريقة؟ يستبعد دلالات الأبناء فيُقرأ «نسخ» مرة واحدة.

الأسطر 177–182

```
      child: Material(
        color: const Color(0xFF173A3E),
        borderRadius: BorderRadius.circular(14),
        clipBehavior: Clip.antiAlias,
        child: InkWell(
          onTap: onTap,
```

ماذا يفعل؟ `Material` بلون `173A3E` وزوايا 14 و `InkWell` .

لماذا بهذه الطريقة؟ لون التصميم لزر على البطاقة الداكنة. `Material` ليظهر أثر اللمس.

الأسطر 183–188

```
          child: SizedBox(
            height: 44,
            child: Row(
              mainAxisAlignment: MainAxisAlignment.center,
              spacing: 8,
              children: [
```

ماذا يفعل؟ `SizedBox` ارتفاعه 44 و `Row` في الوسط بمسافة 8.

لماذا بهذه الطريقة؟ 44 هو الحد الأدنى المريح للمس.

الأسطر 189–193

```
                Icon(icon, size: 18, color: const Color(0xFF7ED3B2)),
                Text(
                  label,
                  style: AppTextStyles.label14.copyWith(color: AppColors.light),
                ),
```

ماذا يفعل؟ أيقونة خضراء فاتحة ونص بلون `light`.

لماذا بهذه الطريقة؟ تباين جيد على الخلفية الداكنة.

الأسطر 194–201

```
              ],
            ),
          ),
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_CodeButton`.

الأسطر 203–208

```
class _WhyCard extends StatelessWidget {
  const _WhyCard();

  @override
  Widget build(BuildContext context) {
    return RizmaCard(
```

ماذا يفعل؟ ودجت `_WhyCard` ثابت: `RizmaCard` بحشوة 16/14.

لماذا بهذه الطريقة؟ ثابت `const` لأن لا حالة له.

الأسطر 209–213

```
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 14),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.stretch,
        spacing: 4,
        children: [
```

ماذا يفعل؟ `Column` ممتد بمسافة 4.

لماذا بهذه الطريقة؟ عنوان ثم ثلاثة صفوف.

السطر 214

```
214           Text(ArStrings.codeWhyTitle, style: AppTextStyles.section18),
```

ماذا يفعل؟ عنوان «لماذا هذا الرمز؟».

لماذا بهذه الطريقة؟ نمط `section18`.

الأسطر 215–220

```
215           const _WhyRow(
216             icon: Icons.phone_android_rounded,
217             iconColor: AppColors.teal,
218             tint: AppColors.tealTint,
219             text: ArStrings.codeWhy1,
220           ),
```

ماذا يفعل؟ سبب 1: «انقل تقدّمك إلى جهاز جديد» بأيقونة هاتف خضراء.

لماذا بهذه الطريقة؟ أهم استعمال للرمز.

الأسطر 221–226

```
221           const _WhyRow(
222             icon: Icons.group_outlined,
223             iconColor: AppColors.blue,
224             tint: AppColors.blueTint,
225             text: ArStrings.codeWhy2,
226           ),
```

ماذا يفعل؟ سبب 2: «معلمك يضيفك إلى مجموعته» بأيقونة مستخدمين زرقاء.

لماذا بهذه الطريقة؟ يخدم رمز الصف لاحقًا.

الأسطر 227–232

```
227           const _WhyRow(
228             icon: Icons.shield_outlined,
229             iconColor: AppColors.purple,
230             tint: AppColors.purpleTint,
231             text: ArStrings.codeWhy3,
232           ),
```

ماذا يفعل؟ سبب 3: «لا يحتاج بريدًا ولا رقم هاتف» بأيقونة درع بنفسجية.

لماذا بهذه الطريقة؟ يؤكد خصوصية الطالب.

الأسطر 233–237

```
233         ],
234       ),
235     );
236   }
237 }
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية `_WhyCard`.

الأسطر 239–245

```
class _WhyRow extends StatelessWidget {
  const _WhyRow({
    required this.icon,
    required this.iconColor,
    required this.tint,
    required this.text,
  });
```

ماذا يفعل؟ ودجت `_WhyRow` ومعاملاته: الأيقونة ولونها وخلفيتها والنص.

لماذا بهذه الطريقة؟ صف واحد يخدم الأسباب الثلاثة.

الأسطر 247–250

```
  final IconData icon;
  final Color iconColor;
  final Color tint;
  final String text;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 252–258

```
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(vertical: 8),
      child: Row(
        spacing: 12,
        children: [
```

ماذا يفعل؟ `build`: حشوة رأسية 8 و`Row` بمسافة 12.

لماذا بهذه الطريقة؟ تباعد الصفوف.

الأسطر 259–266

```
          IconTile(
            iconData: icon,
            iconColor: iconColor,
            color: tint,
            size: 40,
            iconSize: 20,
            radius: 12,
          ),
```

ماذا يفعل؟ مربّع الأيقونة 40.

لماذا بهذه الطريقة؟ نفس مقاسات التصميم.

الأسطر 267–272

```
          Expanded(child: Text(text, style: AppTextStyles.body15)),
        ],
      ),
    );
  }
}
```

ماذا يفعل؟ النص في `Expanded` ثم إغلاق الصنف.

لماذا بهذه الطريقة؟ النص الطويل يلتف على سطرين بدل أن يتجاوز الحافة.

الأسطر 274–277

```
class _ClassGroupRow extends StatelessWidget {
  const _ClassGroupRow({required this.profile});

  final Profile profile;
```

ماذا يفعل؟ ودجت `_ClassGroupRow` ومعامله: الملف.

لماذا بهذه الطريقة؟ يفصل منطق مجموعة الصف.

الأسطر 279–281

```
  @override
  Widget build(BuildContext context) {
    return _InfoRow(
```

ماذا يفعل؟ `build` ويعيد `_InfoRow`.

لماذا بهذه الطريقة؟ نعيد استعمال بطاقة المعلومة نفسها.

الأسطر 282–289

```
      iconData: Icons.group_outlined,
      iconColor: AppColors.amberDark,
      tint: AppColors.amberTint,
      title: ArStrings.codeClassGroup,
      caption: ArStrings.codeClassLine(profile.classCode!),
    );
  }
}
```

ماذا يفعل؟ أيقونة عنبرية، والعنوان «مجموعة الصف» والتوضيح «رمز الصف: GZ-5A» من `classCode`.

لماذا بهذه الطريقة؟ `!` آمنة لأننا لا نبني هذا الودجت إلا إن كان `classCode != null`. لا نخترع اسم مجموعة لا نعرفه (التصميم يكتب «الخامس (أ)» وهو مثال).

الأسطر 291–299

```
/// بطاقة معلومة: أيقونة ملوّنة وسطر عنوان وسطر توضيح.
class _InfoRow extends StatelessWidget {
  const _InfoRow({
    required this.iconData,
    required this.iconColor,
    required this.tint,
    required this.title,
    required this.caption,
  });
```

ماذا يفعل؟ تعليق وصنف `_InfoRow` ومعاملاته: الأيقونة ولونها وخلفيتها والعنوان والتوضيح.

لماذا بهذه الطريقة؟ بطاقة معلومة تُستعمل للمزامنة والمجموعة.

الأسطر 301–305

```
  final IconData iconData;
  final Color iconColor;
  final Color tint;
  final String title;
  final String caption;
```

ماذا يفعل؟ الحقول.

لماذا بهذه الطريقة؟ غير قابلة للتغيير.

الأسطر 307–314

```
  @override
  Widget build(BuildContext context) {
    return DecoratedBox(
      decoration: BoxDecoration(
        color: AppColors.card,
        borderRadius: BorderRadius.circular(18),
        border: Border.all(color: AppColors.line),
      ),
```

ماذا يفعل؟ `DecoratedBox` : `build` أبيض بزوايا 18 وحدّ رفيع.

لماذا بهذه الطريقة؟ مثل بطاقات التصميم.

الأسطر 315–319

```
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 12),
        child: Row(
          spacing: 12,
          children: [
```

ماذا يفعل؟ حشوة 14/12 و `Row` بمسافة 12.

لماذا بهذه الطريقة؟ تباعد الصف.

الأسطر 320–327

```
            IconTile(
              iconData: iconData,
              iconColor: iconColor,
              color: tint,
              size: 40,
              iconSize: 20,
              radius: 12,
            ),
```

ماذا يفعل؟ مربّع الأيقونة 40.

لماذا بهذه الطريقة؟ الحجم نفسه في كل مكان.

الأسطر 328–331

```
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
```

ماذا يفعل؟ `Expanded` ثم `Column` محاذاة البداية.

لماذا بهذه الطريقة؟ يأخذ العرض المتبقي.

الأسطر 332–333

```
                  Text(title, style: AppTextStyles.body15),
                  Text(caption, style: AppTextStyles.caption13),
```

ماذا يفعل؟ سطر العنوان بنمط `body15` وسطر التوضيح بنمط `caption13`.

لماذا بهذه الطريقة؟ تسلسل بصري واضح.

الأسطر 334–342

```
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

ماذا يفعل؟ إغلاق العناصر والصنف.

لماذا بهذه الطريقة؟ نهاية الملف.